

Informe técnico

Procesamiento distribuido de imágenes (Erlang + Scheme)

Alejandro Jiménez Segura
José Mario Castro Cruz

septiembre de 2026

Índice

1. Arquitectura del sistema	3
2. División de la imagen	4
3. Modelo de concurrencia	5
4. Comunicación entre Erlang y Scheme	6
5. Protocolo de comunicación	7
6. Implementación del filtro	8
7. Manejo de bordes	9
8. Manejo de errores	10
9. Resultados experimentales	11
10. Análisis de speedup y eficiencia	14
10.1. ¿Se tuvieron en cuenta las fortalezas y debilidades de cada lenguaje?	15
11. Limitaciones del sistema	16

1. Arquitectura del sistema

El sistema está dividido en varias etapas y utiliza Erlang como el encargado de organizar el trabajo y Scheme como el encargado de aplicar los filtros a la imagen. La idea principal es que cada lenguaje tenga una tarea diferente: Erlang se encarga de leer la imagen, dividirla, repartir el trabajo y juntar los resultados, mientras que Scheme recibe cada región, la procesa y devuelve el resultado.

Flujo general

1. **Lectura.** El módulo `ppm.erl` lee el archivo PPM y obtiene el ancho, alto, valor máximo y los píxeles de la imagen. Los píxeles se almacenan en una estructura de filas y columnas.
2. **Preparación.** En `main/6`, Erlang valida los parámetros recibidos, como el número de regiones, el tamaño del filtro y el tipo de borde. También calcula el radio necesario para los filtros que utilizan vecinos.
3. **División.** La función `dividirBandas/2` divide la imagen en varias bandas horizontales. Por ejemplo, una imagen de 100 filas dividida en cuatro regiones puede quedar aproximadamente como $\{0,25\}$, $\{25,50\}$, $\{50,75\}$ y $\{75,100\}$.
4. **Distribución.** Erlang crea los procesos trabajadores mediante `spawn/3`. Cada trabajador ejecuta `trabajar/3`, abre un puerto hacia Scheme y envía la región que debe procesar. Esto permite que varias regiones sean procesadas al mismo tiempo.
5. **Procesamiento.** Scheme recibe la información de una región, aplica el filtro seleccionado y devuelve los píxeles procesados. Por ejemplo, si se selecciona **GAUSSIAN**, Scheme aplica el filtro gaussiano a los píxeles de la región recibida.
6. **Reconstrucción.** Cuando Erlang recibe todos los resultados, `imgServer/3` los organiza por número de región. Luego `reconstruir/1` vuelve a unir las bandas para formar la imagen completa y `ppm:escribirPPM/5` escribe el resultado en el archivo de salida.

Ejemplo del flujo

Supongamos que se procesa una imagen de 100 filas utilizando cuatro regiones. Erlang primero divide la imagen en cuatro bandas y crea los procesos correspondientes. Cada proceso recibe una banda y la envía a Scheme:

Imagen completa

```
|
v
+-----+-----+-----+-----+
| B1    | B2    | B3    | B4    |
+-----+-----+-----+-----+
|        |        |        |        |
v        v        v        v
Scheme Scheme Scheme Scheme
|        |        |        |
+-----+-----+-----+
|
v
```

Imagen reconstruida

2. División de la imagen

Se eligió dividir la imagen en bandas horizontales. Cada banda conserva todo el ancho de la imagen y recibe un subconjunto de las filas.

La función `dividirBandas/2` calcula una distribución equilibrada de las filas. Si el alto no es divisible exactamente entre N , las primeras bandas reciben una fila adicional hasta distribuir el residuo.

Ejemplo

Para una imagen de 10 filas y $N = 3$, una distribución posible es:

Banda 1: filas 0..3

Banda 2: filas 4..6

Banda 3: filas 7..9

Ventajas

- La división es sencilla.
- Todas las regiones mantienen el ancho completo.
- El halo solo necesita extenderse verticalmente.
- No es necesario agregar columnas adicionales a izquierda y derecha.
- La reconstrucción consiste principalmente en concatenar las filas de las regiones en su orden original.

Alternativa: cuadrícula 2D

Una alternativa sería dividir la imagen en bloques mediante una cuadrícula de filas y columnas. La cuadrícula podría ofrecer un reparto más flexible del trabajo, pero aumenta la complejidad del protocolo. Cada región podría necesitar halo en cuatro direcciones: arriba, abajo, izquierda y derecha.

En una división horizontal, en cambio, el halo requerido para una convolución solamente se agrega arriba y abajo. Por esta razón, las bandas horizontales ofrecen una relación favorable entre simplicidad y funcionalidad para este proyecto.

3. Modelo de concurrencia

La concurrencia se implementa mediante procesos independientes de Erlang. La función `lanzarTodos/3` inicia los trabajos y cada proceso ejecuta `trabajar/3`.

El proceso trabajador:

- Recibe la región asignada.
- Construye la petición para Scheme.
- Abre un puerto hacia Scheme.
- Envía la petición.
- Espera la respuesta.
- Envía el resultado a `imgServer/3`.

La comunicación entre procesos utiliza mensajes de Erlang, por ejemplo:

```
Server ! {resultado, Idx, R}
```

El servidor utiliza un `receive` para procesar los mensajes que van llegando. Por lo tanto, el programa no ejecuta las regiones mediante una secuencia simple de llamadas.

```
procesar región 1
    esperar
procesar región 2
    esperar
procesar región 3
```

En su lugar, varias regiones pueden estar siendo procesadas simultáneamente.

Control de concurrencia:

El proyecto incorpora además una cola de regiones pendientes. No se lanzan necesariamente todas las instancias de Scheme al mismo tiempo. El cupo inicial se calcula con base en los schedulers disponibles y se limita al número total de regiones.

Cuando una región termina, `imgServer/3` puede lanzar otra región pendiente. Esto evita crear una cantidad excesiva de procesos externos y reduce el riesgo de saturar CPU y memoria.

4. Comunicación entre Erlang y Scheme

La comunicación se realiza mediante puertos de Erlang hacia procesos externos de Scheme. La apertura del puerto utiliza:

```
trabajar(Server, Idx, Peticion) ->
Port = open_port({spawn, "racket " ++ ?RACKET_SCRIPT}, [binary, exit_status, {packet,
4}]),
port_command(Port, list_to_binary(Peticion)),
Server ! {resultado, Idx, esperar(Port)}.
```

Framing de transporte

Se utiliza `{packet,4}`. Esto permite que el puerto conozca la longitud de cada paquete mediante un campo de longitud de 4 bytes. De esta manera, aunque el contenido de la petición sea texto, el transporte sigue teniendo un límite claro de mensaje.

Formato de petición

Una petición contiene líneas con la siguiente estructura:

```
FILTER GAUSSIAN
PARAMS 3
DIMS 400 303
HALO 0 1 0 1
BORDE EXTENDER
PIXELS ...
```

Los campos principales son:

FILTER	Identifica el filtro solicitado.
PARAMS	Contiene parámetros del filtro, por ejemplo el tamaño del kernel.
DIMS	Ancho y alto de la región enviada.
HALO	Cantidad de filas adicionales alrededor de la región real.
BORDE	Estrategia para los bordes: extender o ceros.
PIXELS	Lista plana de valores RGB.

Formato de respuesta

Una respuesta correcta tiene la forma:

```
OK
DIMS 400 300
PIXELS ...
```

Si ocurre un error, Scheme puede devolver:

```
ERROR mensaje
```

¿Por qué una instancia de Scheme por región?

El diseño permite que cada región sea independiente. Cada proceso Erlang abre su propio proceso Scheme, envía una petición, recibe una respuesta y finaliza la instancia. Esto simplifica el aislamiento entre regiones y hace que un fallo de una instancia no implique necesariamente la pérdida del procesamiento de las demás regiones. Erlang puede detectar el fallo y reintentar la región. La desventaja es el costo de iniciar Scheme para cada región. Un pool de procesos Scheme persistentes podría reducir este costo, pero agregaría complejidad al diseño.

5. Protocolo de comunicación

La comunicación entre Erlang y Scheme se realiza mediante puertos. Erlang se encarga de abrir el proceso de Scheme y enviarle la información de la región de la imagen que debe procesar. Después, espera una respuesta con los píxeles ya procesados.

Cada trabajador de Erlang utiliza `open_port/2` para iniciar Scheme y luego utiliza `port_command/2` para enviar la petición. La información enviada contiene varios campos que permiten a Scheme conocer cómo debe procesar la región.

Un ejemplo de la estructura de una petición es:

```
FILTER GAUSSIAN
PARAMS 3
DIMS 100 27
HALO 0 1 0 1
BORDE extender
PIXELS ...
```

Los campos tienen las siguientes funciones:

- **FILTER** indica el filtro que se debe aplicar, por ejemplo **GAUSSIAN**, **SHARPEN** o **GRAYSCALE**.
- **PARAMS** contiene el parámetro utilizado por el filtro. Por ejemplo, para el filtro gaussiano puede indicar el tamaño del filtro.
- **DIMS** indica las dimensiones de la región que se está enviando.
- **HALO** indica la información adicional que se incluye alrededor de la región para los filtros que necesitan utilizar píxeles vecinos.
- **BORDE** indica cómo se deben tratar los píxeles que están fuera de los límites de la imagen.
- **PIXELS** contiene los valores RGB de los píxeles que Scheme debe procesar.

Scheme recibe esta información, procesa la región y devuelve una respuesta. Cuando el procesamiento termina correctamente, la respuesta comienza con **OK** e incluye las dimensiones y los píxeles resultantes. Erlang utiliza `parseRespuesta/1` para interpretar esta información y convertir los valores recibidos nuevamente en tripletas RGB.

Además, la comunicación utiliza un tamaño de paquete de 4 bytes para indicar la longitud de los datos enviados. Esto permite que Erlang y Scheme sepan dónde comienza y termina cada mensaje. En Scheme, las funciones `leer-entero32` y `escribir-paquete` se encargan de leer y escribir este tamaño antes de procesar el contenido del mensaje.

De esta manera, el protocolo permite separar la comunicación de la lógica del procesamiento. Erlang se encarga de enviar las regiones y recibir los resultados, mientras que Scheme se concentra en aplicar el filtro correspondiente.

6. Implementación del filtro

El archivo `scheme/filtro` contiene el procesamiento de las regiones. Este se implementa de forma genérica. Para cada píxel se obtienen los vecinos definidos por el kernel y se calcula un producto punto mediante `pp`.

```
(define pp
  (lambda (v w)
    (apply + (map * v w))))
```

Kernel gaussiano 3x3

Para `ksize = 3`, el kernel generado es:

```
1 2 1
2 4 2
1 2 1
```

El proyecto no deja el kernel completamente fijo. Scheme genera el kernel a partir del triángulo de Pascal. Para un tamaño `K` impar se toma la fila `K-1` y se calcula su producto externo consigo misma. Esto permite utilizar, por ejemplo, kernels `3×3`, `5×5`, `7×7`, etc., sin tener que modificar los coeficientes manualmente.

Filtro	Descripción	Halo
Gaussian	Difuminado mediante convolución gaussiana	Sí
Sharpen	Realce de bordes mediante kernel <code>3×3</code>	Sí
Grayscale	Conversión RGB a escala de grises	No
Invert	Invierte cada canal: <code>255 - valor</code>	No
Threshold	Convierte la imagen en blanco/negro según un umbral	No
Brightness	Suma un desplazamiento a cada canal	No

Como añadido, los filtros `grayscale`, `invert`, `threshold` y `brightness` no requieren vecinos, ya que en estos los píxeles se procesan de forma independiente.

7. Manejo de bordes

Las convoluciones necesitan consultar píxeles vecinos. En los límites de una imagen algunos de esos vecinos pueden quedar fuera de sus dimensiones reales.

El proyecto implementa dos estrategias:

Extensión

Cuando se solicita un píxel fuera de los límites, se utiliza el píxel válido más cercano. Por ejemplo, si se solicita una fila menor que cero, se utiliza la primera fila. Si se solicita una columna mayor que el ancho, se utiliza la última columna.

Ceros

Los píxeles que quedan fuera de la imagen se consideran: **(0, 0, 0)**

Las dos estrategias se pueden seleccionar desde la línea de comandos.

Halo

El halo resuelve la dependencia entre bandas vecinas. Para un kernel de radio r , una banda recibe r filas adicionales arriba y r filas adicionales abajo. Por ejemplo, para un kernel 3×3 :

$$\text{radio} = (3 - 1) / 2 = 1$$

Por tanto, una banda recibe una fila adicional arriba y otra abajo cuando esas filas existen dentro de la imagen.

Para un kernel 5×5 :

$$\text{radio} = (5 - 1) / 2 = 2$$

Recorte del halo

Scheme procesa toda la región recibida, incluido el halo. Después elimina las filas adicionales mediante **recortar-halo**, de modo que solo devuelve los píxeles pertenecientes a la banda original.

8. Manejo de errores

El sistema cuenta con diferentes mecanismos para detectar y manejar errores durante el procesamiento de la imagen. La mayor parte de este manejo se realiza en Erlang, ya que es el encargado de distribuir el trabajo y recibir los resultados de cada región.

- **Validación de parámetros.** Antes de comenzar el procesamiento se validan los parámetros recibidos por el programa. Por ejemplo, se verifica que el número de regiones sea un entero positivo y que no sea mayor que la cantidad de filas de la imagen. También se valida que el tamaño del filtro sea impar y mayor o igual a 3, y que el tipo de borde sea `extender` o `ceros`. Si algún parámetro no es válido, el programa muestra un mensaje de error y termina.
- **Errores durante el procesamiento de una región.** Cada región es procesada por un trabajador de Erlang. Si Scheme devuelve un error, el trabajador envía esta información al servidor mediante un mensaje. La función `registrar/4` se encarga de revisar el error y determinar qué hacer.
- **Reintento de una región.** Si una región falla, el sistema intenta procesarla nuevamente una vez. Esto permite recuperar algunos errores temporales sin tener que detener inmediatamente todo el procesamiento. Si la región vuelve a fallar, se marca como un error definitivo.
- **Control de tiempo de espera.** El trabajador espera una respuesta de Scheme durante un tiempo máximo de 30 segundos. Si no recibe una respuesta dentro de ese tiempo, se genera un error de tipo `timeout`. La región puede ser reintentada una vez antes de considerarse fallida.
- **Respuestas inválidas de Scheme.** Erlang utiliza `parseRespuesta/1` para interpretar la respuesta recibida. Si la respuesta no tiene el formato esperado, se considera inválida y se registra como un error. De esta manera, el programa no continúa utilizando datos que no pudo interpretar correctamente.
- **Errores definitivos.** Si una región falla después del reintento, el sistema guarda el error junto con el número de la región. Cuando todas las regiones han terminado, `hayError/1` revisa los resultados. Si existe alguna región con error, no se escribe el archivo de salida.
- **Evitar imágenes incompletas.** Una de las medidas principales del sistema es que no se genera una imagen de salida si alguna de las regiones falló definitivamente. Esto evita producir un archivo que parezca válido pero que tenga partes sin procesar.

En general, el manejo de errores permite que el sistema detecte problemas en los parámetros, en la comunicación con Scheme y durante el procesamiento de las regiones. Además, el uso de reintentos permite recuperarse de algunos errores temporales, mientras que la validación final evita generar una imagen incompleta cuando un error no puede ser solucionado.

9. Resultados experimentales

Para tener un control de los resultados se va a utilizar una imagen de tamaño 100×100 y se aplicará el filtro gaussiano mediante el siguiente comando en la terminal de Linux Mint.

```
./scripts/benchmark.sh test_images/checkerboard_100x100.ppm gaussian "1 2 4 8"
```

La estructura del comando es la siguiente:

Comando	Función
./scripts/benchmark.sh	Ejecuta un script que realiza las mediciones pedidas en el proyecto
test_images/checkerboard_100x100.ppm	Selecciona la imagen a la que se le va a aplicar el filtro
gaussian	Filtro seleccionado
"1 2 4 8"	Cantidad de procesos en paralelo

Especificación de la computadora de la prueba

- Sistema Operativo: Linux Mint 22.3 - Cinnamon 64-bit
- Procesador: AMD Ryzen 7 7800X3D 8-Core Processor $\times 8$
- Tarjeta gráfica: NVIDIA Corporation TU117 GeForce GTX 1650
- Memoria: 15.4 GB

Tamaño	Procesos	T_p (ms)	S_p	E_p	C_p (ms)
400×300	1	1565	1,00	1,00	1565
400×300	2	978	1,60	0,80	1956
400×300	4	748	2,09	0,52	2992
400×300	8	693	2,26	0,28	5544

Tamaño	Procesos	T_p (ms)	S_p	E_p	C_p (ms)
100×100	1	360	1,00	1,00	360
100×100	2	188	1,91	0,96	376
100×100	4	191	1,88	0,47	764
100×100	8	297	1,21	0,15	2376

Speedup:

El speedup permite comparar el tiempo de ejecución utilizando p procesos con respecto al tiempo utilizando un solo proceso.

$$S_p = \frac{T_1}{T_p} \quad (1)$$

donde T_1 representa el tiempo de ejecución utilizando un proceso y T_p representa el tiempo de ejecución utilizando p procesos.

Eficiencia:

La eficiencia permite observar qué tan bien se aprovechan los procesos utilizados en la ejecución.

$$E_p = \frac{S_p}{p} \quad (2)$$

donde S_p representa el speedup obtenido y p representa la cantidad de procesos utilizados.

Coste:

El coste representa el tiempo total de procesamiento considerando la cantidad de procesos utilizados.

$$C_p = p \times T_p \quad (3)$$

donde C_p representa el coste, p la cantidad de procesos y T_p el tiempo de ejecución con p procesos.

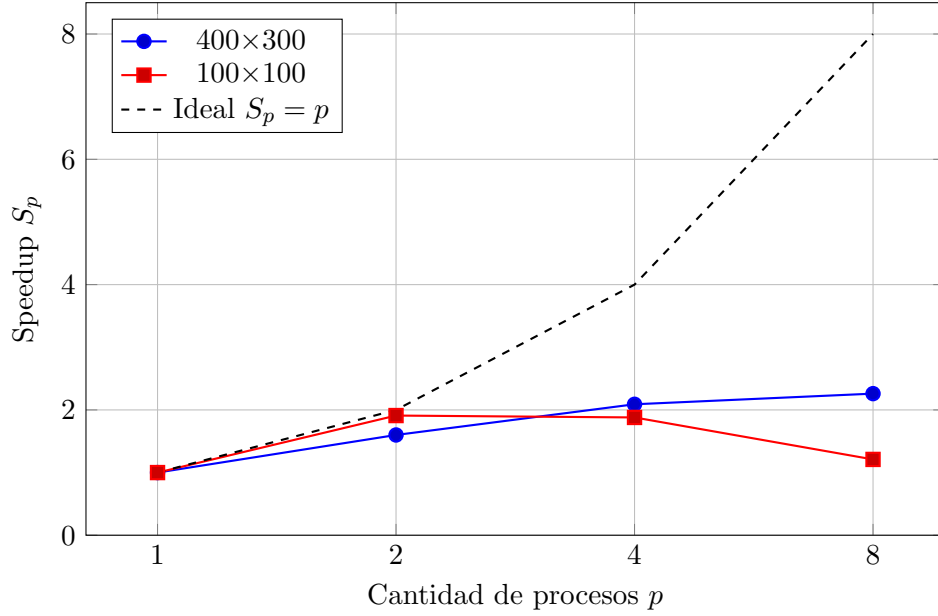


Figura 1: Speedup medido en función de la cantidad de procesos. La imagen de 400x300 presenta un aumento del speedup al utilizar más procesos, aunque se aleja del comportamiento ideal. En la imagen de 100x100, el mayor speedup se obtiene con 2 procesos y disminuye al utilizar 4 y 8 procesos.

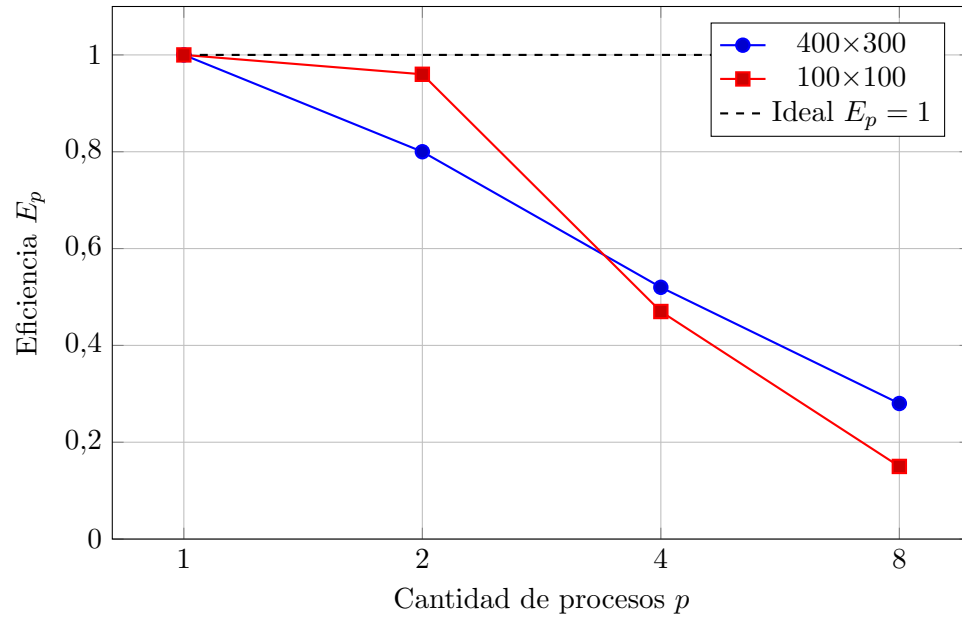


Figura 2: Eficiencia medida en función de la cantidad de procesos. En ambas imágenes la eficiencia disminuye al aumentar la cantidad de procesos. La disminución es más marcada en la imagen de 100×100 , donde los costos adicionales asociados a la paralelización tienen un mayor efecto sobre el resultado.

10. Análisis de speedup y eficiencia

Los resultados muestran que aumentar el número de procesos no garantiza automáticamente una reducción del tiempo de ejecución. En la imagen de 100×100 el tiempo baja de 360 ms con un proceso a 188 ms con dos, pero de 2 a 4 procesos prácticamente no cambia ($188 \text{ ms} \rightarrow 191 \text{ ms}$), lo que sugiere que el trabajo útil ya se agotó y el proceso extra no aporta nada. Al subir a 8 procesos el tiempo empeora de forma clara, llegando a 297 ms, un valor incluso peor que el de un solo proceso. En la imagen de 400×300 , en cambio, se observa una reducción progresiva del tiempo al aumentar la cantidad de procesos, pasando de 1565 ms con uno a 693 ms con ocho.

Este comportamiento puede explicarse por diferentes costos adicionales asociados con la ejecución paralela. Entre los principales factores se encuentran:

- **Creación y administración de procesos:** Erlang debe crear y coordinar los procesos encargados de procesar las diferentes bandas. Aunque la concurrencia permite distribuir el trabajo, la creación, planificación y administración de una mayor cantidad de procesos introduce un costo adicional. Sin embargo, el costo más significativo no está en los procesos ligeros de Erlang, que son baratos, sino en el arranque de una instancia nueva de Scheme por banda: cada `open_port` inicia el runtime de Scheme, carga Swindle y compila el módulo antes de poder procesar el primer píxel. Ese costo fijo se paga una vez por región, por lo que crece linealmente con la cantidad de bandas.
- **Latencia de comunicación e I/O entre Erlang y Scheme:** cada proceso debe comunicarse con un programa Scheme mediante un puerto. Esto implica enviar la información de la región, esperar el procesamiento y recibir el resultado. Además, los datos de los píxeles viajan como texto y deben convertirse de número a texto en Erlang y de texto a número en Scheme. El tiempo empleado en estas operaciones puede reducir la ganancia obtenida mediante la paralelización.
- **Lectura, división, copia y escritura de datos:** antes del procesamiento es necesario leer la imagen, dividirla en regiones y preparar los datos que serán enviados a los procesos. Las regiones incluyen información adicional para manejar el halo de los filtros que utilizan vecinos, lo que aumenta el volumen enviado. Al finalizar, los resultados deben ser recibidos y reconstruidos para generar la imagen de salida. Estas operaciones forman parte del tiempo total, pero no corresponden directamente al cálculo del filtro y no se paralelizan.
- **Tamaño de la imagen:** una imagen pequeña contiene menos trabajo que distribuir. En este caso, los costos asociados con la creación de procesos, la comunicación y el manejo de los datos pueden representar una parte importante del tiempo total. Una imagen más grande proporciona una mayor cantidad de trabajo que puede ser distribuido entre los procesos, por lo que la paralelización puede aprovecharse mejor.

En conjunto, el speedup observado se aleja del ideal porque una fracción creciente del tiempo total corresponde a trabajo que no se paraleliza: el arranque del intérprete de Scheme, la serialización de texto y la lectura y escritura de archivos. Para la imagen de 100×100 esa fracción domina, de modo que agregar procesos sólo aumenta el costo sin repartir mejor el trabajo. Para la imagen de 400×300 el trabajo útil por banda es suficiente para compensar parcialmente ese costo, por lo que sí se observa una mejora al aumentar la cantidad de procesos, aunque cada vez más lejos del speedup ideal.

10.1. ¿Se tuvieron en cuenta las fortalezas y debilidades de cada lenguaje?

El reparto de responsabilidades se diseñó de acuerdo con las características de ambos lenguajes.

Erlang

- **Fortalezas:** concurrencia, paso de mensajes, tolerancia a fallos y coordinación.
- **Limitaciones:** no está especializado en cómputo numérico intensivo de píxeles.

Scheme

- **Fortalezas:** recursión, manipulación de listas y estructuras.
- **Limitaciones:** no se utiliza como coordinador de concurrencia; el procesamiento numérico sobre listas puede ser costoso.

11. Limitaciones del sistema

- **Se crea una instancia de Scheme por cada banda.** Cada región de la imagen abre una nueva instancia de Scheme para realizar el procesamiento. Esto agrega un tiempo adicional, ya que el programa debe iniciarse y cargar el archivo de Scheme cada vez.
- **Los datos de los píxeles se envían como texto.** Los valores de los píxeles deben convertirse a texto para enviarlos de Erlang a Scheme y luego volver a convertirse en números. Esto aumenta el trabajo necesario para procesar cada región.
- **La imagen se divide únicamente en bandas horizontales.** El sistema divide la imagen por filas. Esto facilita la distribución del trabajo, pero limita otras formas de dividir la imagen, como utilizar bloques más pequeños.
- **El uso de halo aumenta la cantidad de datos enviados.** Los filtros que necesitan conocer los píxeles vecinos reciben filas adicionales alrededor de cada región. Esto es necesario para aplicar correctamente algunos filtros, pero hace que se envíen más píxeles de los que pertenecen directamente a la región.
- **La transposición utiliza una sola región.** Cuando se utiliza el filtro `transponer`, el programa fuerza el número de regiones a uno. Esto limita el procesamiento en paralelo de este filtro.
- **El sistema depende de tiempos de espera.** Si un trabajador no responde, Erlang espera hasta que se cumpla el tiempo establecido antes de considerar que ocurrió un error. Esto puede aumentar el tiempo total de una ejecución cuando existe algún problema con una región.
- **Se procesa un filtro por ejecución.** El programa aplica un solo filtro cada vez. Si se quieren aplicar varios filtros consecutivamente, es necesario realizar varias ejecuciones del programa.
- **La imagen completa se mantiene en memoria.** El sistema carga toda la imagen antes de comenzar el procesamiento. Esto funciona bien para imágenes de tamaño normal, pero podría consumir bastante memoria si se utilizan imágenes mucho más grandes.